

Distributed Machine Learning and Big Data

Sourangshu Bhattacharya

Dept. of Computer Science and Engineering,
IIT Kharagpur.

<http://cse.iitkgp.ac.in/~sourangshu/>

August 21, 2015

Outline

- 1 Machine Learning and Big Data
 - Support Vector Machines
 - Stochastic Sub-gradient descent
- 2 Distributed Optimization
 - ADMM
 - Convergence
 - Distributed Loss Minimization
 - Results
 - Development of ADMM
- 3 Applications and extensions
 - Weighted Parameter Averaging
 - Fully-distributed SVM

What is Big Data ?

- 6 Billion web queries per day.
- 10 Billion display advertisements per day.
- 30 Billion text ads per day.
- 150 Million credit card transactions per day.
- 100 Billion emails per day.

Machine Learning on Big Data

- Classification - Spam / No Spam - 100B emails.
- Multi-label classification - image tagging - 14M images 10K tags.
- Regression - CTR estimation - 10B ad views.
- Ranking - web search - 6B queries.
- Recommendation - online shopping - 1.7B views in the US.

Classification example

- Email spam classification.
- Features ($\mathbf{u}_i$): Vector of counts of all words.
- No. of Features (d): Words in vocabulary ($\sim 100,000$).
- No. of non-zero features: 100.
- No. of emails per day: **100 M**.
- Size of training set using 30 days data: **6 TB** (assuming 20 B per data)
- Time taken to read the data once: **41.67 hrs** (at 20 MB per second)
- Solution: **use multiple computers**.



Big Data Paradigm

- 3V's - Volume, Variety, Velocity.

- Distributed system.

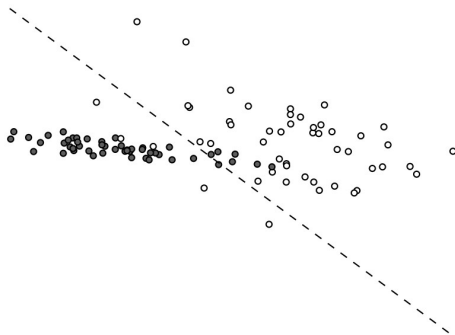
- Chance of failure:

Computers	1	10	100
Chance of a failure in an hour	0.01	0.09	0.63

- Communication efficiency - Data locality.
- Many systems: Hadoop, Spark, Graphlab, etc.
- **Goal:** Implement Machine Learning algorithms on Big data systems.

Binary Classification Problem

- A set of labeled datapoints $(S) = \{(\mathbf{u}_i, v_i), i = 1, \dots, n\}$, $\mathbf{u}_i \in \mathbb{R}^d$ and $v_i \in \{+1, -1\}$
- Linear Predictor function: $v = \text{sign}(\mathbf{x}^T \mathbf{u})$
- Error function: $E = \sum_{i=1}^n \mathbf{1}(v_i \mathbf{x}^T \mathbf{u}_i \leq 0)$



Logistic Regression

- Probability of v is given by:

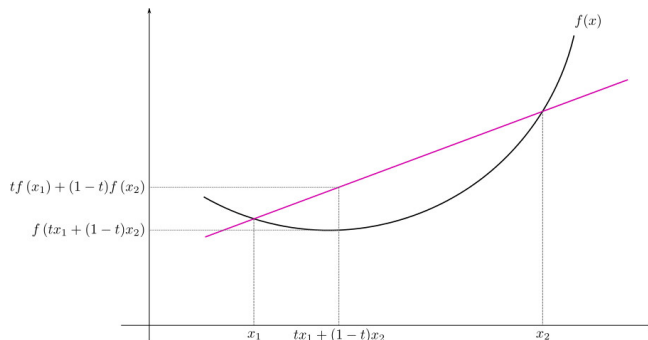
$$P(v|\mathbf{u}, \mathbf{x}) = \sigma(v\mathbf{x}^T\mathbf{u}) = \frac{1}{1 + e^{-v\mathbf{x}^T\mathbf{u}}}$$

- Learning problem is:
Given dataset $\mathcal{S}$, estimate $\mathbf{x}$.
- Maximizing the regularized log likelihood:

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} \sum_{i=1}^n \log(1 + e^{-v_i\mathbf{x}^T\mathbf{u}_i}) + \frac{\lambda}{2}\mathbf{x}^T\mathbf{x}$$

Convex Function

- f is a Convex function: $f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$



Convex Optimization

Convex optimization problem

$$\begin{array}{ll}\text{minimize}_x & f(x) \\ \text{subject to:} & g_i(x) \leq 0, \forall i = 1, \dots, k\end{array}$$

where:

- f, g_i are convex functions.
- For convex optimization problems, local optima are also global optima.

Optimization Algorithm: Gradient Descent

input : Function f , Gradient ∇f

output: Optimal solution w^*

Initialize $w_0 \leftarrow 0$, $k \leftarrow 0$

while $|\nabla f_k| > \epsilon$ **do**

 Compute $\alpha_k \leftarrow \text{linesearch}(f, -\nabla f_k, w_k)$

 Set $w_{k+1} \leftarrow w_k - \alpha_k \nabla f_k$

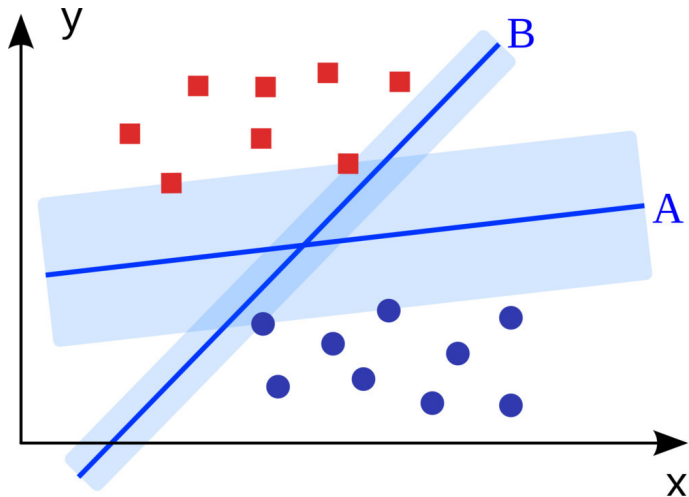
 Evaluate ∇f_{k+1}

$k \leftarrow k + 1$

end

$w^* \leftarrow w_k$

Classification Problem



SVM

- Separating hyperplane: $\mathbf{x}^T \mathbf{u} = 0$
- Parallel hyperplanes (developing margin): $\mathbf{x}^T \mathbf{u} = \pm 1$
- Margin (perpendicular distance between parallel hyperplanes): $\frac{2}{\|\mathbf{x}\|}$
- Correct classification of training datapoints: $v_i \mathbf{x}^T \mathbf{u}_i \geq 1, \forall i$
- Allowing error (slack), ξ_i : $v_i \mathbf{x}^T \mathbf{u}_i \geq 1 - \xi_i, \forall i$
- Max-margin formulation:

$$\min_{\mathbf{x}, \xi} \frac{1}{2} \|\mathbf{x}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to: $v_i \mathbf{x}^T \mathbf{u}_i \geq 1 - \xi_i, \xi_i \geq 0 \forall i = 1, \dots, n$

SVM: dual

- Lagrangian:

$$L = \frac{1}{2} \mathbf{x}^T \mathbf{x} + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - \mathbf{v}_i \mathbf{x}^T \mathbf{u}_i) + \sum_{i=1}^n \mu_i \xi_i$$

- Dual problem: $(\mathbf{x}^*, \alpha^*, \mu^*) = \max_{\alpha, \mu} \min_{\mathbf{x}} L(\mathbf{x}, \alpha, \mu)$
- For strictly convex problem, primal and dual solutions are same (Strong duality).
- KKT conditions:

$$\mathbf{x} = \sum_{i=1}^n \alpha_i \mathbf{v}_i \mathbf{u}_i$$

$$C = \alpha_i + \mu_i$$

SVM: dual

The dual problem:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^{n, n} \alpha_i \alpha_j \mathbf{v}_i \mathbf{v}_j^T \mathbf{u}_i \mathbf{u}_j$$

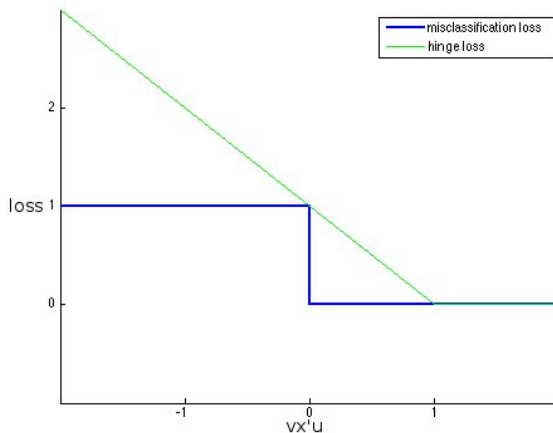
subject to: $0 \leq \alpha_i \leq C, \forall i$

- The dual is a quadratic programming problem in n variables.
- Can be solved even if kernel function, $k(\mathbf{u}_i, \mathbf{u}_j) = \mathbf{u}_i^T \mathbf{u}_j$ are given. Dimension agnostic.
- Many efficient algorithms exist for solving it, e.g. SMO (Platt99).
- Worst case complexity is $O(n^3)$, usually $O(n^2)$.

SVM

A more compact form: $\min_{\mathbf{x}} \sum_{i=1}^n \max(0, 1 - v_i \mathbf{x}^T \mathbf{u}_i) + \lambda \|\mathbf{x}\|_2^2$

Or: $\min_{\mathbf{x}} \sum_{i=1}^n l(\mathbf{x}, \mathbf{u}_i, v_i) + \lambda \Omega(\mathbf{x})$



Multi-class classification

- There are m classes. $v_i \in \{1, \dots, m\}$
- Most popular scheme: $v_i = \operatorname{argmax}_{v \in \{1, \dots, m\}} \mathbf{x}_v^T \mathbf{u}_i$
- Given example $(\mathbf{u}_i, v_i)$, $\mathbf{x}_{v_i}^T \mathbf{u}_i \geq \mathbf{x}_j^T \mathbf{u}_i \forall j \in \{1, \dots, m\}$
- Using a margin of at least 1, loss
$$l(\mathbf{u}_i, v_i) = \max_{j \in \{1, \dots, v_i-1, v_i+1, \dots, m\}} \{0, 1 - (\mathbf{x}_{v_i}^T \mathbf{u}_i - \mathbf{x}_j^T \mathbf{u}_i)\}$$
- Given dataset $\mathcal{D}$, solve the problem

$$\min_{\mathbf{x}_1, \dots, \mathbf{x}_m} \sum_{i \in \mathcal{D}} l(\mathbf{u}_i, v_i) + \lambda \sum_{j=1}^m \|\mathbf{x}_j\|^2$$

- This can be extended to many settings e.g. sequence labeling, learning to rank, etc.

General Learning Problems

- Support Vector Machines:

$$\min_{\mathbf{x}} \sum_{i=1}^n \max\{0, 1 - v_i \mathbf{x}^T \mathbf{u}_i\} + \lambda \|\mathbf{x}\|_2^2$$

- Logistic Regression:

$$\min_{\mathbf{x}} \sum_{i=1}^n \log(1 + \exp(-v_i \mathbf{x}^T \mathbf{u}_i)) + \lambda \|\mathbf{x}\|_2^2$$

- General form:

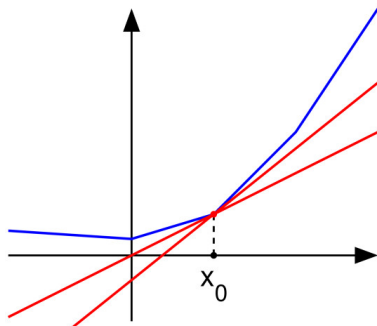
$$\min_{\mathbf{x}} \sum_{i=1}^n l(\mathbf{x}, \mathbf{u}_i, v_i) + \lambda \Omega(\mathbf{x})$$

- l : loss function, Ω : regularizer.

Sub-gradient Descent

- Sub-gradient for a non-differentiable convex function f at a point $\mathbf{x}_0$ is a vector $\mathbf{v}$ such that:

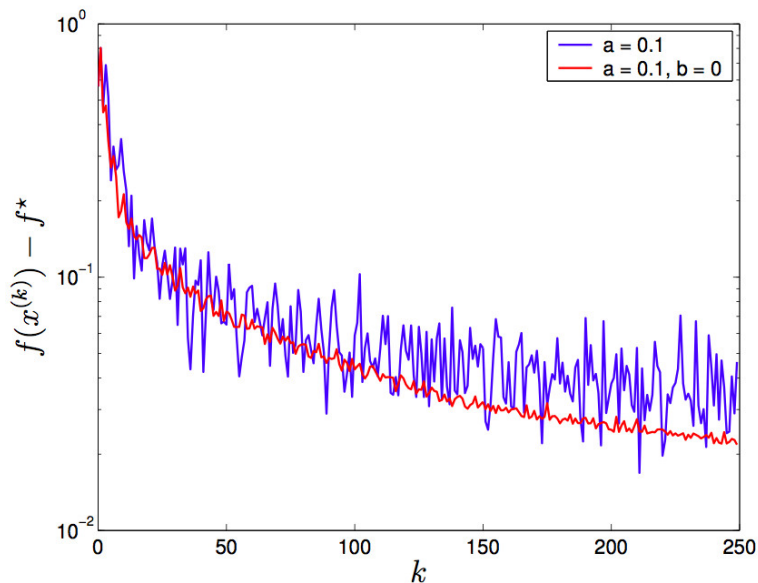
$$f(\mathbf{x}) - f(\mathbf{x}_0) \geq \mathbf{v}^T (\mathbf{x} - \mathbf{x}_0)$$



Sub-gradient Descent

- Randomly initialize $\mathbf{x}_0$
- Iterate $\mathbf{x}_k = \mathbf{x}_{k-1} - t_k g(\mathbf{x}_{k-1})$, $k = 1, 2, 3, \dots$. Where g is a sub-gradient of f .
- $t_k = \frac{1}{\sqrt{k}}$.
- $x_{best}(k) = \min_{i=1, \dots, k} f(x_i)$

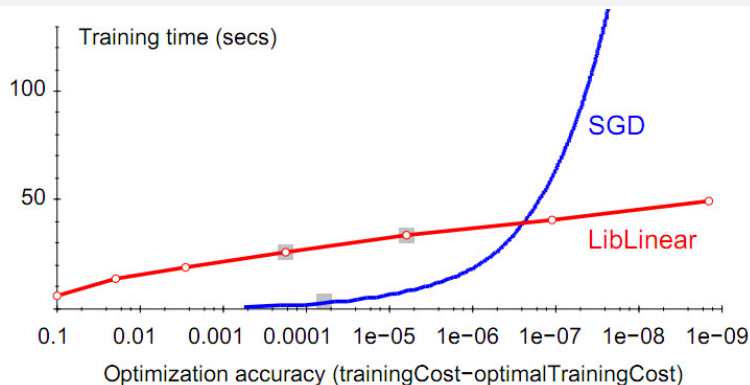
Sub-gradient Descent



Stochastic Sub-gradient Descent

- Convergence rate is: $O(\frac{1}{\sqrt{k}})$.
- Each iteration takes $O(n)$ time.
- Reduce time by calculating the gradient using a subset of examples - stochastic subgradient.
- Inherently serial.
- Typical $O(\frac{1}{\epsilon^2})$ behaviour.

Stochastic Sub-gradient Descent



Distributed gradient descent

- Divide the dataset into m parts. Each part is processed on one computer. Total m .
- There is one central computer.
- All computers can communicate with the central computer via network.
- Define $loss(\mathbf{x}) = \sum_{j=1}^m \sum_{i \in C_j} l_i(\mathbf{x}) + \lambda \Omega(\mathbf{x})$, where $l_i(\mathbf{x}) = l(\mathbf{x}, \mathbf{u}_i, v_i)$
- The gradient (in case of differentiable loss):

$$\nabla loss(\mathbf{x}) = \sum_{j=1}^m \nabla \left(\sum_{i \in C_j} l_i(\mathbf{x}) \right) + \lambda \nabla \Omega(\mathbf{x})$$

- Compute $\nabla l_j(\mathbf{x}) = \sum_{i \in C_j} \nabla l_i(\mathbf{x})$ on the j^{th} computer. Communicate to central computer.

Distributed gradient descent

- Compute $\nabla \text{loss}(\mathbf{x}) = \sum_{j=1}^m \nabla l_j(\mathbf{x}) + \Omega(\mathbf{x})$ at the central computer.
- The gradient descent update: $\mathbf{x}^{k+1} = \mathbf{x}^k - \alpha \nabla \text{loss}(\mathbf{x})$.
- α chosen by a line search algorithm (distributed).
- For non-differentiable loss functions, we can use distributed sub-gradient descent algorithm.
 - Slow for most practical problems.
 - For achieving ϵ tolerance,
 - Gradient descent (Logistic regression): $O(1/\epsilon)$ iterations.
 - Sub-gradient descent (Stochastic Sub-gradient descent): $O(\frac{1}{\epsilon^2})$ iterations.

Alternating Direction Method of Multipliers

Problem

$$\begin{aligned} & \text{minimize}_{x,z} \quad f(x) + g(z) \\ & \text{subject to:} \quad Ax + Bz = c \end{aligned}$$

Algorithm

Iterate till convergence:

- $x^{k+1} = \operatorname{argmin}_x f(x) + \frac{\rho}{2} \|Ax + Bz^k - c + u^k\|_2^2$
- $z^{k+1} = \operatorname{argmin}_z g(z) + \frac{\rho}{2} \|Ax^{k+1} + Bz - c + u^k\|_2^2$
- $u^{k+1} = u^k + Ax^{k+1} + Bz^{k+1} - c$

Stopping criteria

- Stop when primal and dual residuals small:

$$\|r^k\|_2 \leq \epsilon^{pri} \text{ and } \|s^k\|_2 \leq \epsilon^{dual}$$

Hence, $\|r^k\|_2 \rightarrow 0$ and $\|s^k\|_2 \rightarrow 0$ as $k \rightarrow \infty$

Observations

- x - update requires solving an optimization problem

$$\min_x f(x) + \frac{\rho}{2} \|Ax - v\|_2^2$$

with, $v = Bz^k - c + u^k$

- Similarly for z -update.
- Sometimes has a closed form.
- ADMM is a meta optimization algorithm.

Convergence of ADMM

- Assumption 1: Functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $g : \mathbb{R}^m \rightarrow \mathbb{R}$ are closed, proper and convex.
 - Same as assuming $\text{epi} f = \{(x, t) \in \mathbb{R}^n \times \mathbb{R} \mid f(x) \leq t\}$ is closed and convex.
- Assumption 2: The unaugmented Lagrangian $L_0(x, y, z)$ has a saddle point (x^*, z^*, y^*) :

$$L_0(x^*, z^*, y) \leq L_0(x^*, z^*, y^*) \leq L_0(x, z, y^*)$$

Convergence of ADMM

- Primal residual: $r = Ax + Bz - c$
- Optimal objective: $p^* = \inf_{x,z} \{f(x) + g(z) | Ax + Bz = c\}$
- Convergence results:
 - Primal residual convergence: $r^k \rightarrow 0$ as $k \rightarrow \infty$
 - Dual residual convergence: $s^k \rightarrow 0$ as $k \rightarrow \infty$
 - Objective convergence: $f(x) + g(z) \rightarrow p^*$ as $k \rightarrow \infty$
 - Dual variable convergence: $y^k \rightarrow y^*$ as $k \rightarrow \infty$

Decomposition

- If f is separable:

$$f(x) = f_1(x_1) + \cdots + f_N(x_N), \quad x = (x_1, \dots, x_N)$$

- A is conformably block separable; i.e. $A^T A$ is block diagonal.
- Then, x -update splits into N parallel updates of x_i

Consensus Optimization

- Problem:

$$\min_x f(x) = \sum_{i=1}^N f_i(x)$$

- ADMM form:

$$\begin{aligned} \min_{x_i, z} \quad & \sum_{i=1}^N f_i(x_i) \\ \text{s.t.} \quad & x_i - z = 0, \quad i = 1, \dots, N \end{aligned}$$

- Augmented lagrangian:

$$L_\rho(x_1, \dots, x_N, z, y) = \sum_{i=1}^N (f_i(x_i) + y_i^T (x_i - z) + \frac{\rho}{2} \|x_i - z\|_2^2)$$

Consensus Optimization

- ADMM algorithm:

$$x_i^{k+1} = \operatorname{argmin}_{x_i} (f_i(x_i) + y_i^{kT}(x_i - z^k) + \frac{\rho}{2} \|x_i - z^k\|_2^2)$$

$$z^{k+1} = \frac{1}{N} \sum_{i=1}^N (x_i^{k+1} + \frac{1}{\rho} y_i^k)$$

$$y_i^{k+1} = y_i^k + \rho(x_i^{k+1} - z^{k+1})$$

- Final solution is z^k .

Consensus Optimization

- z-update can be written as:

$$z^{k+1} = \bar{x}^{k+1} + \frac{1}{\rho} \bar{y}^{k+1}$$

- Averaging the y-updates:

$$\bar{y}^{k+1} = \bar{y}^k + \rho(\bar{x}^{k+1} - z^{k+1})$$

- Substituting first into second: $\bar{y}^{k+1} = 0$. Hence $z^k = \bar{x}^k$.
- Revised algorithm:

$$x_i^{k+1} = \operatorname{argmin}_{x_i} (f_i(x_i) + y_i^{kT}(x_i - \bar{x}^k) + \frac{\rho}{2} \|x_i - \bar{x}^k\|_2^2)$$

$$y_i^{k+1} = y_i^k + \rho(x_i^{k+1} - \bar{x}^{k+1})$$

- Final solution is z^k .

Distributed Loss minimization

- Problem:

$$\min_x l(Ax - b) + r(x)$$

- Partition A and b by rows:

$$A = \begin{bmatrix} A_1 \\ \vdots \\ A_N \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_N \end{bmatrix},$$

where, $A_i \in \mathbb{R}^{m_i \times m}$ and $b_i \in \mathbb{R}^{m_i}$

- ADMM formulation:

$$\min_{x_i, z} \sum_{i=1}^N l_i(A_i x_i - b_i) + r(z)$$

$$\text{s.t.: } x_i - z = 0, \quad i = 1, \dots, N$$

Distributed Loss minimization

- ADMM solution:

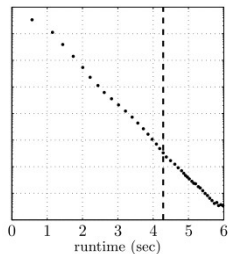
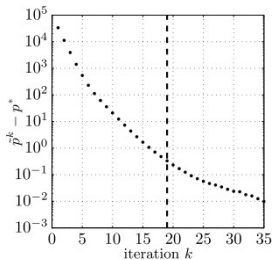
$$x_i^{k+1} = \operatorname{argmin}_{x_i} (l_i(A_i x_i - b_i) + \frac{\rho}{2} \|x_i - z^k + u_i^k\|_2^2)$$

$$z^{k+1} = \operatorname{argmin}_z (r(z) + \frac{N\rho}{2} \|z - \bar{x}^{k+1} + \bar{u}^k\|_2^2)$$

$$u_i^{k+1} = u_i^k + x_i^{k+1} - z^{k+1}$$

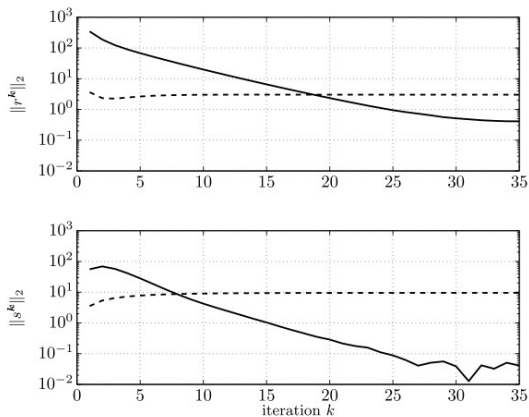
ADMM Results

Logistic Regression using the loss minimization formulation (Boyd et al.):



ADMM Results

Logistic Regression using the loss minimization formulation (Boyd et al.):

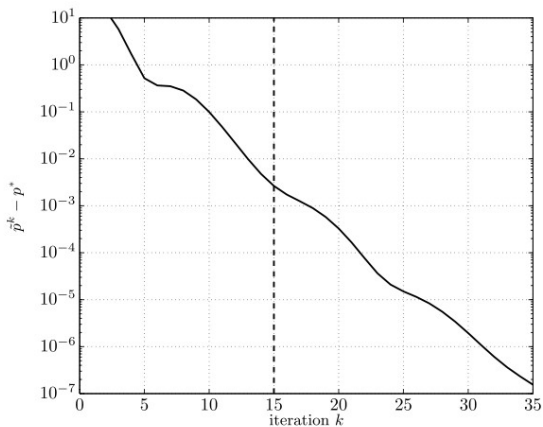


Other Machine Learning Problems

- Ridge Regression.
- Lasso.
- Multi-class SVM.
- Ranking.
- Structured output prediction.

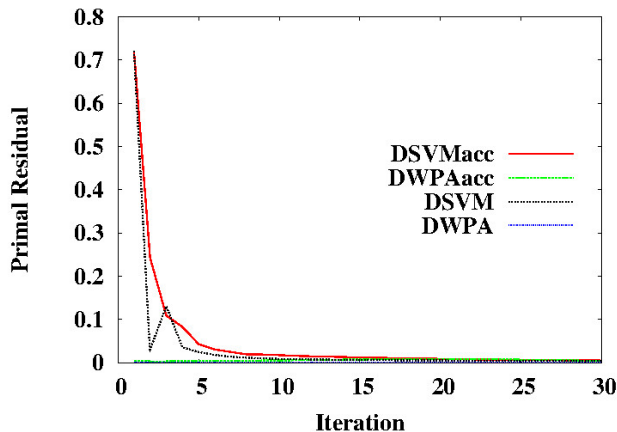
ADMM Results

Lasso Results (Boyd et al.):



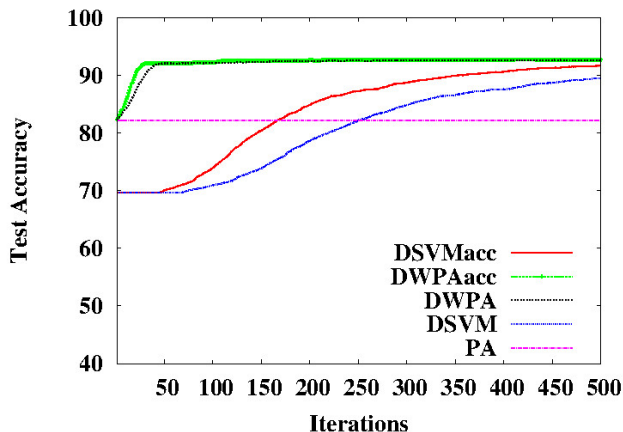
ADMM Results

SVM primal residual:



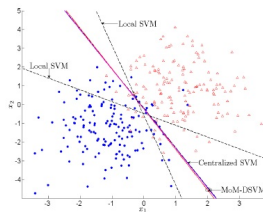
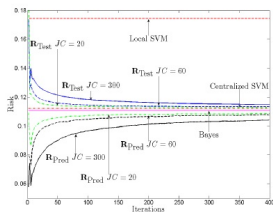
ADMM Results

SVM Accuracy:



Results

Risk and Hyperplane



Dual Ascent

- Convex equality constrained problem:

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to: } & Ax = b \end{aligned}$$

- Lagrangian: $L(x, y) = f(x) + y^T(Ax - b)$
- Dual function: $g(y) = \inf_x L(x, y)$
- Dual problem: $\max_y g(y)$
- Final solution: $x^* = \operatorname{argmin}_x L(x, y)$

Dual Ascent

- Gradient descent for dual problem: $y^{k+1} = y^k + \alpha^k \nabla_{y^k} g(y^k)$
- $\nabla_{y^k} g(y^k) = A\tilde{x} - b$, where $\tilde{x} = \operatorname{argmin}_x L(x, y^k)$
- Dual ascent algorithm:

$$\begin{aligned}x^{k+1} &= \operatorname{argmin}_x L(x, y^k) \\ y^{k+1} &= y^k + \alpha^k (Ax^{k+1} - b)\end{aligned}$$

- Assumptions:
 - $L(x, y^k)$ is strictly convex. Else, the first step can have multiple solutions.
 - $L(x, y^k)$ is bounded below.

Dual Decomposition

- Suppose f is separable:

$$f(x) = f_1(x_1) + \cdots + f_N(x_N), \quad x = (x_1, \dots, x_N)$$

- L is separable in x : $L(x, y) = L_1(x_1, y) + \cdots + L_N(x_N, y) - y^T b$,
where $L_i(x_i, y) = f_i(x_i) + y^T A_i x_i$
- x minimization splits into N separate problems:

$$x_i^{k+1} = \operatorname{argmin}_{x_i} L_i(x_i, y^k)$$

Dual Decomposition

- Dual decomposition:

$$x_i^{k+1} = \operatorname{argmin}_{x_i} L_i(x_i, y^k), \quad i = 1, \dots, N$$

$$y^{k+1} = y^k + \alpha^k \left(\sum_{i=1}^N A_i x_i - b \right)$$

- Distributed solution:
 - Scatter y^k to individual nodes
 - Compute x_i in the i^{th} node (distributed step)
 - Gather $A_i x_i$ from the i^{th} node
- All drawbacks of dual ascent exist

Method of Multipliers

- Make dual ascent work under more general conditions
- Use **augmented Lagrangian**:

$$L_{\rho}(x, y) = f(x) + y^T(Ax - b) + \frac{\rho}{2}\|Ax - b\|_2^2$$

- Method of multipliers:

$$x^{k+1} = \operatorname{argmin}_x L_{\rho}(x, y^k)$$

$$y^{k+1} = y^k + \rho(Ax^{k+1} - b)$$

Methods of Multipliers

- Optimality conditions (for differentiable f):

- Primal feasibility: $Ax^* - b = 0$
- Dual feasibility: $\nabla f(x^*) + A^T y^* = 0$

- Since x^{k+1} minimizes $L_\rho(x, y^k)$

$$\begin{aligned} 0 &= \nabla_x L_\rho(x^{k+1}, y^k) \\ &= \nabla_x f(x^{k+1}) + A^T(y^k + \rho(Ax^{k+1} - b)) \\ &= \nabla_x f(x^{k+1}) + A^T y^{k+1} \end{aligned}$$

- Dual update $y^{k+1} = y^k + \rho(Ax^{k+1} - b)$ makes (x^{k+1}, y^{k+1}) dual feasible
- Primal feasibility is achieved in the limit: $(Ax^{k+1} - b) \rightarrow 0$

Alternating direction method of multipliers

- Problem with applying standard method of multipliers for distributed optimization:
 - there is no problem decomposition even if f is separable.
 - due to square term $\frac{\rho}{2} \|Ax - b\|_2^2$

Alternating direction method of multipliers

- ADMM problem:

$$\begin{aligned} \min_{x,z} \quad & f(x) + g(z) \\ \text{subject to: } & Ax + Bz = c \end{aligned}$$

- Lagrangian:

$$L_\rho(x, z, y) = f(x) + g(z) + y^T(Ax + Bz - c) + \frac{\rho}{2} \|Ax + Bz - c\|_2^2$$

- ADMM:

$$\begin{aligned} x^{k+1} &= \operatorname{argmin}_x L_\rho(x, z^k, y^k) \\ z^{k+1} &= \operatorname{argmin}_z L_\rho(x^{k+1}, z, y^k) \\ y^{k+1} &= y^k + \rho(Ax^{k+1} + Bz^{k+1} - c) \end{aligned}$$

Alternating direction method of multipliers

- Problem with applying standard method of multipliers for distributed optimization:
 - there is no problem decomposition even if f is separable.
 - due to square term $\frac{\rho}{2} \|Ax - b\|_2^2$
- The above technique reduces to method of multipliers if we do joint minimization of x and z
- Since we split the joint x, z minimization step, the problem can be decomposed.

ADMM Optimality conditions

- Optimality conditions (differentiable case):
 - Primal feasibility: $Ax + Bz - c = 0$
 - Dual feasibility: $\nabla f(x) + A^T y = 0$ and $\nabla g(z) + B^T y = 0$
- Since z^{k+1} minimizes $L_\rho(x^{k+1}, z, y^k)$:

$$\begin{aligned} 0 &= \nabla g(z^{k+1}) + B^T y^k + \rho B^T (Ax^{k+1} + Bz^{k+1} - c) \\ &= \nabla g(z^{k+1}) + B^T y^{k+1} \end{aligned}$$

- So, the dual variable update satisfies the second dual feasibility constraint.
- Primal feasibility and first dual feasibility are satisfied asymptotically.

ADMM Optimality conditions

- Primal residual: $r^k = Ax^k + Bz^k - c$
- Since x^{k+1} minimizes $L_\rho(x, z^k, y^k)$:

$$\begin{aligned} 0 &= \nabla f(x^{k+1}) + A^T y^k + \rho A^T (Ax^{k+1} + Bz^k - c) \\ &= \nabla f(x^{k+1}) + A^T (y^k + \rho r^{k+1} + \rho B(z^k - z^{k+1})) \\ &= \nabla f(x^{k+1}) + A^T y^{k+1} + \rho A^T B(z^k - z^{k+1}) \end{aligned}$$

or,

$$\rho A^T B(z^k - z^{k+1}) = \nabla f(x^{k+1}) + A^T y^{k+1}$$

- Hence, $s^{k+1} = \rho A^T B(z^k - z^{k+1})$ can be thought as dual residual.

ADMM with scaled dual variables

- Combine the linear and quadratic terms
 - Primal feasibility: $Ax + Bz - c = 0$
 - Dual feasibility: $\nabla f(x) + A^T y = 0$ and $\nabla g(z) + B^T y = 0$
- Since z^{k+1} minimizes $L_\rho(x^{k+1}, z, y^k)$:

$$\begin{aligned} 0 &= \nabla g(z^{k+1}) + B^T y^k + \rho B^T (Ax^{k+1} + Bz^{k+1} - c) \\ &= \nabla g(z^{k+1}) + B^T y^{k+1} \end{aligned}$$

- So, the dual variable update satisfies the second dual feasibility constraint.
- Primal feasibility and first dual feasibility are satisfied asymptotically.

Distributed Support Vector Machines

- Training dataset partitioned into M partitions ($\mathcal{S}_m$, $m = 1, \dots, M$).
- Each partition has L datapoints: $\mathcal{S}_m = \{(\mathbf{x}_{ml}, y_{ml})\}$, $l = 1, \dots, L$.
- Each partition can be processed locally on a single computer.
- Distributed SVM training problem [?]:

$$\begin{aligned} \min_{\mathbf{w}_m, \mathbf{z}} \quad & \sum_{m=1}^M \sum_{l=1}^L \text{loss}(\mathbf{w}_m; (\mathbf{x}_{ml}, y_{ml})) + r(\mathbf{z}) \\ \text{s.t.} \quad & \mathbf{w}_m - \mathbf{z} = 0, m = 1, \dots, M, l = 1, \dots, L \end{aligned}$$

Parameter Averaging

- Parameter averaging, also called “mixture weights” proposed in [?], for logistic regression.
- Results hold true for SVMs with suitable sub-derivative.
- Locally learn SVM on $\mathcal{S}_m$:

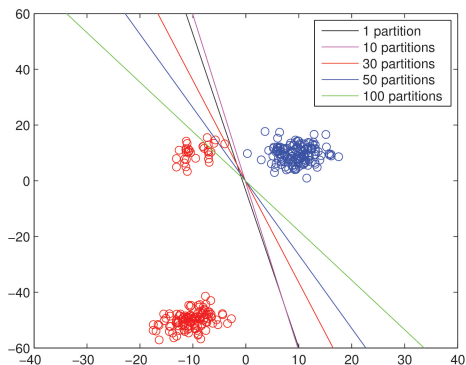
$$\hat{\mathbf{w}}_m = \operatorname{argmin}_{\mathbf{w}} \frac{1}{L} \sum_{l=1}^L \operatorname{loss}(\mathbf{w}; \mathbf{x}_{ml}, y_{ml}) + \lambda \|\mathbf{w}\|^2, \quad m = 1, \dots, M$$

- The final SVM parameter is given by:

$$\mathbf{w}_{PA} = \frac{1}{M} \sum_{m=1}^M \hat{\mathbf{w}}_m$$

Problem with Parameter Averaging

PA with varying number of partitions - Toy dataset.



Weighted Parameter Averaging

- Final hypothesis is a weighted sum of the parameters $\hat{\mathbf{w}}_m$.

$$\mathbf{w} = \sum_{m=1}^M \beta_m \mathbf{w}_m$$

- Also proposed in [?].
- How to get β_m ?
- Notation: $\boldsymbol{\beta} = [\beta_1, \dots, \beta_M]^T$, $\mathbf{W} = [\hat{\mathbf{w}}_1, \dots, \hat{\mathbf{w}}_M]$

$$\mathbf{w} = \mathbf{W}\boldsymbol{\beta}$$

Weighted Parameter Averaging

- Find the optimal set of weights β which attains the lowest regularized hinge loss:

$$\min_{\beta, \xi} \lambda \|\mathbf{W}\beta\|^2 + \frac{1}{ML} \sum_{m=1}^M \sum_{i=1}^L \xi_{mi}$$

$$\begin{aligned} \text{subject to: } y_{mi}(\beta^T \mathbf{W}^T \mathbf{x}_{mi}) &\geq 1 - \xi_{mi}, \quad \forall i, m \\ \xi_{mi} &\geq 0, \quad \forall m = 1, \dots, M, i = 1, \dots, L \end{aligned}$$

- $\hat{\mathbf{W}}$ is a pre-computed parameter.

Distributed Weighted Parameter Averaging

- Distributed version of primal weighted parameter averaging:

$$\begin{aligned} \min_{\gamma_m, \beta} \quad & \frac{1}{ML} \sum_{m=1}^M \sum_{l=1}^L \text{loss}(\hat{W}\gamma_m; \mathbf{x}_{ml}, y_{ml}) + r(\beta) \\ \text{s.t.} \quad & \gamma_m - \beta = 0, \quad m = 1, \dots, M, \end{aligned}$$

- $r(\beta) = \lambda \|\hat{W}\beta\|^2$, γ_m weights for m^{th} computer, β consensus weight.

Distributed Weighted Parameter Averaging

- Distributed algorithm using ADMM:

$$\gamma_m^{k+1} := \operatorname{argmin}_{\gamma} (\operatorname{loss}(\mathbf{A}_i \gamma) + (\rho/2) \|\gamma - \beta^k + \mathbf{u}_m^k\|_2^2)$$

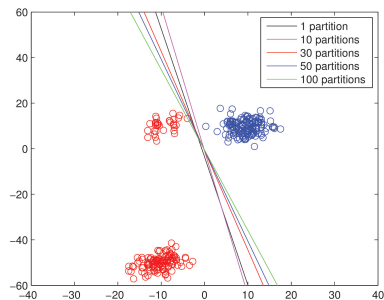
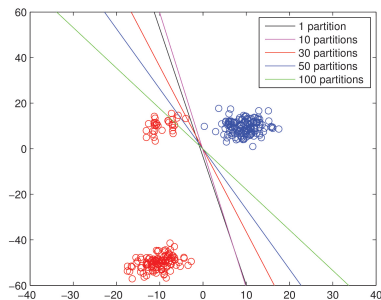
$$\beta^{k+1} := \operatorname{argmin}_{\beta} (r(\beta) + (M\rho/2) \|\beta - \bar{\gamma}^{k+1} - \bar{\mathbf{u}}^k\|_2^2)$$

$$\mathbf{u}_m^{k+1} = \mathbf{u}_m^k + \gamma_m^{k+1} - \beta^{k+1}.$$

- $\mathbf{u}_m$ are the scaled Lagrange multipliers, $\bar{\gamma} = \frac{1}{M} \sum_{m=1}^M \gamma_m$ and $\bar{\mathbf{u}} = \frac{1}{M} \sum_{m=1}^M \mathbf{u}_m$.

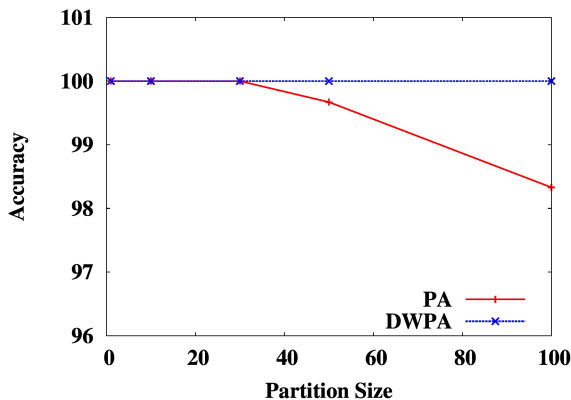
Toy Dataset - PA and WPA

PA (left) and WPA (right) with varying number of partitions - Toy dataset.



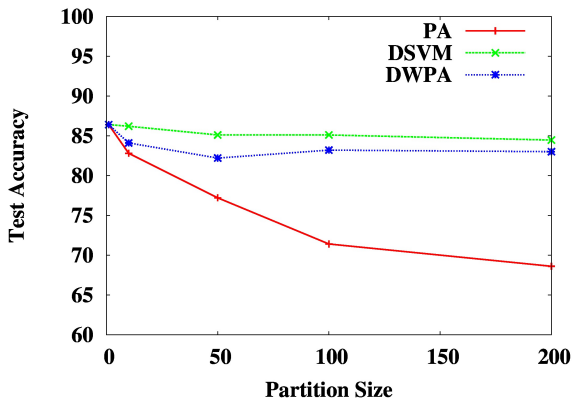
Toy Dataset - PA and WPA

Accuracy of PA and WPA with varying number of partitions - Toy dataset.



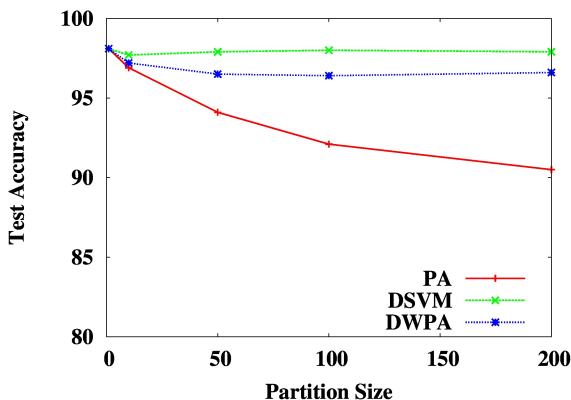
Real World Datasets

Epsilon (2000 features, 6000 datapoints) test set accuracy with varying number of partitions.



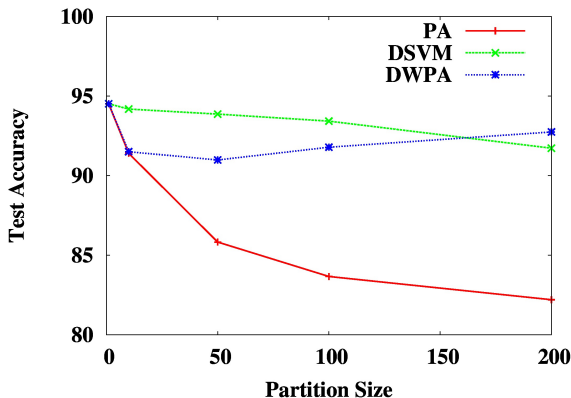
Real World Datasets

Gisette (5000 features, 6000 datapoints) test set accuracy with varying number of partitions.



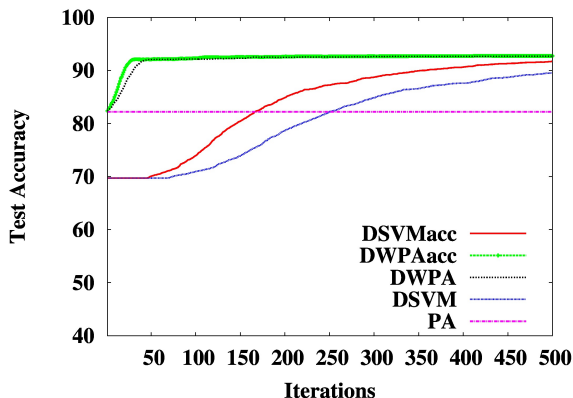
Real World Datasets

Real-sim (20000 features, 3000 datapoints) test set accuracy with varying number of partitions.



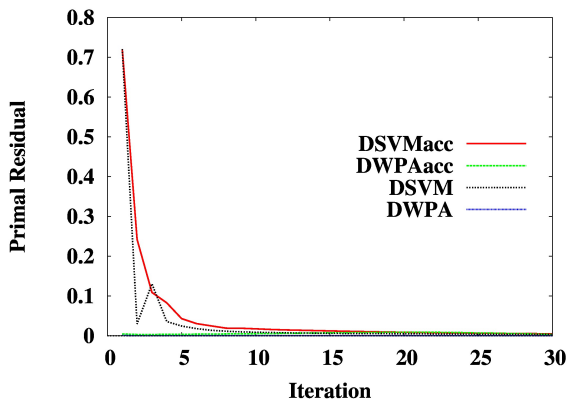
Real World Datasets

Convergence of test accuracy with iterations (200 partitions).



Real World Datasets

Convergence of primal residual with iterations (200 partitions).



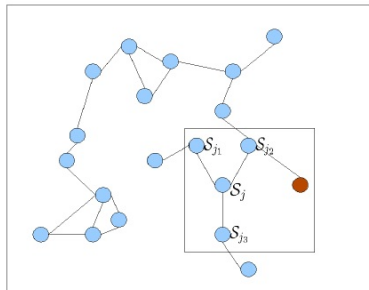
Distributed SVM on Arbitrary Network

Motivations:

- Sensor Networks.
- Corporate networks.
- Privacy.

Assumptions:

- Data is available at nodes of network
- Communication is possible only along edges of the network.



Distributed SVM on Arbitrary Network

- SVM optimization problem:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{j=1}^J \sum_{n=1}^{n_j} \xi_{jn}$$

$$\text{s.t.: } y_{jn}(w^t x_{jn} + b) \geq 1 - \xi_{jn}, \quad \forall j \in J, n = 1, \dots, N_j$$

$$\xi_{jn} \geq 0, \quad \forall j \in J, n = 1, \dots, N_j$$

- Node j has a copy of w_j, b_j . Distributed formulation:

$$\min_{\{w_j, b_j, \xi_{jn}\}} \frac{1}{2} \sum_{j=1}^J \|w_j\|^2 + JC \sum_{j=1}^J \sum_{n=1}^{n_j} \xi_{jn}$$

$$\text{s.t.: } y_{jn}(w_j^t x_{jn} + b) \geq 1 - \xi_{jn}, \quad \forall j \in J, n = 1, \dots, N_j$$

$$\xi_{jn} \geq 0, \quad \forall j \in J, n = 1, \dots, N_j$$

$$w_j = w_i, \quad \forall j, i \in \mathcal{B}_j$$

Algorithm

- Using $\mathbf{v}_j = [\mathbf{w}_j^T \mathbf{b}_j]^T$, $\mathbf{X}_j = [[\mathbf{x}_{j1}, \dots, \mathbf{x}_{jN_j}]^T \mathbf{1}_j]$ and $\mathbf{Y}_j = \text{diag}([y_{j1}, \dots, y_{jN_j}])$:

$$\begin{aligned} \min_{\{\mathbf{v}_j, \xi_{jn}, \omega_{ji}\}} \quad & \frac{1}{2} \sum_{j=1}^J r(\mathbf{v}_j) + JC \sum_{j=1}^J \sum_{n=1}^{n_j} \xi_{jn} \\ \text{s.t.:} \quad & \mathbf{Y}_j \mathbf{X}_j \mathbf{v}_j \geq \mathbf{1} - \bar{\xi}_j, \quad \forall j \in J \\ & \bar{\xi}_j \geq 0, \quad \forall j \in J \\ & \mathbf{v}_j = \omega_{ji}, \quad \mathbf{v}_i = \omega_{ji}, \quad \forall j, i \in \mathcal{B}_j \end{aligned}$$

- Surrogate augmented Lagrangian:

$$\begin{aligned} L(\{\mathbf{v}_j\}, \{\bar{\xi}_j\}, \{\omega_{ji}\}, \{\alpha_{ijk}\}) = & \frac{1}{2} \sum_{j=1}^J r(\mathbf{v}_j) + JC \sum_{j=1}^J \sum_{n=1}^{n_j} \xi_{jn} \\ & + \sum_{j=1}^J \sum_{i \in \mathcal{B}_j} (\alpha_{ij1}^T (\mathbf{v}_j - \omega_{ji}) + \alpha_{ij2}^T (\mathbf{v}_i - \omega_{ji})) + \eta \sum_{i \in \mathcal{B}_j} (\|\mathbf{v}_j - \omega_{ji}\|^2 + \|\mathbf{v}_i - \omega_{ji}\|^2) \end{aligned}$$

Algorithm

- ADMM based algorithm:

$$\{v_j^{t+1}, \xi_{jn}^{t+1}\} = \operatorname{argmin}_{\{v_j, \bar{\xi}_j\} \in \mathcal{W}} L(\{v_j\}, \{\bar{\xi}_j\}, \{\omega_{ji}^t\}, \{\alpha_{ijk}^t\})$$

$$\{\omega_{ji}^{t+1}\} = \operatorname{argmin}_{\omega_{ji}} L(\{v_j\}^{t+1}, \{\bar{\xi}_j^{t+1}\}, \{\omega_{ji}\}, \{\alpha_{ijk}^t\})$$

$$\alpha_{ji1}^{t+1} = \alpha_{ji1}^t + \eta(v_j^{t+1} - \omega_{ji}^{t+1})$$

$$\alpha_{ji2}^{t+1} = \alpha_{ji2}^t + \eta(\omega_{ji}^{t+1} - v_i^{t+1})$$

- From the second equation:

$$\omega_{ji}^{t+1} = \frac{1}{2\eta}(\alpha_{ji1}^t - \alpha_{ji2}^t) + \frac{1}{2}(v_j^{t+1} + v_i^{t+1})$$

Algorithm

- Hence:

$$\alpha_{ji1}^{t+1} = \frac{1}{2}(\alpha_{ji1}^t + \alpha_{ji2}^t) + \frac{\eta}{2}(v_j^{t+1} - v_i^{t+1})$$

$$\alpha_{ji2}^{t+1} = \frac{1}{2}(\alpha_{ji1}^t + \alpha_{ji2}^t) + \frac{\eta}{2}(v_j^{t+1} - v_i^{t+1})$$

- Substituting $\omega_{ji}^{t+1} = \frac{1}{2}(v_j^{t+1} + v_i^{t+1})$ into surrogate augmented lagrangian, the third term becomes:

$$\sum_{j=1}^J \sum_{i \in \mathcal{B}_j} \alpha_{ij1}^T (v_j - v_i) = \sum_{j=1}^J \sum_{i \in \mathcal{B}_j} v_j^T (\alpha_{ji1}^t - \alpha_{ij1}^t)$$

- Substitute $\alpha_j^t = \sum_{i \in \mathcal{B}_j} (\alpha_{ji1}^t - \alpha_{ij1}^t)$

Algorithm

- The final algorithm:

$$\{\mathbf{v}_j^{t+1}, \xi_{jn}^{t+1}\} = \operatorname{argmin}_{\{\mathbf{v}_j, \bar{\xi}_j\} \in \mathcal{W}} L(\{\mathbf{v}_j\}, \{\bar{\xi}_j\}, \{\alpha_j^t\})$$
$$\alpha_j^{t+1} = \alpha_j^t + \frac{\eta}{2} \sum_{i \in \mathcal{B}_j} (\mathbf{v}_j^{t+1} - \mathbf{v}_i^{t+1})$$

Thank you !

Questions ?